

Sistema de Detección de Ruta más Corta Entre Edificios UPB

Sebastian Pico Afanador
sebastian.pico.2022@upb.edu.co

Carlos Felipe Sarmiento Avellaneda
carlosf.sarmiento@upb.edu.co

David Mattias Barbosa Vargas
david.barbosa.2023@upb.edu.co

Facultad de Ingeniería de Sistemas e Informática

1. Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades y comportamientos que el sistema debe ofrecer al usuario final. Se derivan directamente de los criterios de aceptación definidos para el proyecto.

ID	Requisito	Descripción
RF-01	Cálculo de la ruta más corta	El sistema debe calcular la ruta más corta entre un punto de origen y un punto de destino dentro del campus, aplicando el algoritmo de Dijkstra sobre un grafo ponderado.
RF-02	Selección de origen y destino	El sistema debe permitir al usuario seleccionar el punto de origen y el punto de destino mediante una interfaz gráfica, ya sea a través de un mapa interactivo o una lista desplegable.
RF-03	Búsqueda por lugar específico	El sistema debe permitir al usuario buscar directamente un lugar (por ejemplo, "biblioteca" o "auditorio menor") sin necesidad de conocer previamente el edificio en el que se encuentra.
RF-04	Mapeo interno de lugares por edificio	El sistema debe mantener un registro que asocie cada lugar específico con el edificio correspondiente, de modo que pueda resolver automáticamente la ubicación de un lugar buscado.
RF-05	Opción de accesibilidad	El sistema debe ofrecer una opción que permita excluir rutas con escaleras, priorizando caminos alternativos con rampas o ascensores para usuarios con movilidad reducida.
RF-06	Visualización de la ruta calculada	El sistema debe mostrar gráficamente en pantalla la ruta calculada, incluyendo el origen, el destino y los puntos intermedios del recorrido.

ID	Requisito	Descripción
RF-07	Actualización dinámica del grafo	El sistema debe permitir agregar o eliminar conexiones (aristas) entre nodos del grafo, para reflejar bloqueos temporales, cierres o cambios en la infraestructura del campus.
RF-08	Persistencia de la información	El sistema debe almacenar y cargar la información del grafo (edificios, caminos, lugares y sus conexiones) desde un archivo, de modo que los datos no se pierdan entre ejecuciones.
RF-09	Gestión de errores de ruta	El sistema debe informar al usuario cuando no exista una ruta disponible entre el origen y el destino seleccionados (por ejemplo, por bloqueos o desconexión de nodos).

2. Requerimientos No Funcionales

Los requerimientos no funcionales especifican atributos de calidad y restricciones bajo las cuales el sistema debe operar.

ID	Requisito	Descripción
RNF-01	Usabilidad	La interfaz gráfica debe ser intuitiva y comprensible para usuarios sin conocimientos técnicos, con retroalimentación visual clara ante cada acción.
RNF-02	Rendimiento	El cálculo de la ruta más corta debe ejecutarse en un tiempo menor a 2 segundos, incluso al trabajar con grafos de tamaño medio (varios cientos de nodos y aristas).
RNF-03	Portabilidad	Al estar desarrollado en Java, el sistema debe poder ejecutarse en cualquier sistema operativo compatible con la máquina virtual de Java (JVM), sin necesidad de recompilación.
RNF-04	Mantenibilidad	El código debe estar organizado siguiendo principios de programación orientada a objetos (POO), en clases y paquetes claramente separados, y documentado para facilitar su mantenimiento y extensión.
RNF-05	Escalabilidad	La arquitectura del sistema debe permitir incorporar nuevos edificios, caminos o lugares al grafo sin requerir cambios estructurales significativos en el código.
RNF-06	Confiabilidad	El sistema debe manejar adecuadamente situaciones excepcionales, como rutas inexistentes, nodos desconectados o datos de entrada inválidos, sin detener su ejecución de forma inesperada.
RNF-07	Compatibilidad	La interfaz desarrollada en JavaFX debe funcionar correctamente en los equipos de escritorio (Windows) utilizados por los estudiantes, profesores y visitantes de la UPB.
RNF-08	Documentación	El sistema debe contar con documentación técnica clara y estructurada (arquitectura, algoritmo, manual de uso) comprensible para otros desarrolladores.

3. Marco Teórico

3.1 Teoría de Grafos

Un grafo es una estructura matemática compuesta por un conjunto de vértices (o nodos) y un conjunto de aristas (o conexiones) que relacionan pares de vértices. En el contexto de este proyecto, los vértices representan los edificios, entradas y lugares del campus de la UPB, mientras que las aristas representan los caminos o senderos que los conectan.

Un grafo ponderado es aquel en el que cada arista tiene asociado un valor numérico o "peso", que puede representar la distancia, el tiempo de recorrido o el costo asociado a desplazarse entre dos nodos. Este tipo de grafo es fundamental para el proyecto, ya que permite calcular no solo si existe un camino entre dos puntos, sino cuál de ellos es el más eficiente.

Adicionalmente, un grafo puede ser dirigido (las conexiones tienen un sentido único) o no dirigido (las conexiones pueden recorrerse en ambos sentidos). Para este proyecto se utilizará, en general, un grafo no dirigido, ya que los caminos del campus pueden recorrerse en ambos sentidos, salvo que se identifiquen restricciones particulares.

3.2 Representación de Grafos en Memoria

Existen dos formas comunes de representar un grafo en un programa: la matriz de adyacencia y la lista de adyacencia. La matriz de adyacencia utiliza una tabla de tamaño $V \times V$ (donde V es el número de vértices) para indicar la existencia y el peso de las conexiones entre cada par de nodos; es sencilla de implementar, pero poco eficiente en memoria cuando el grafo es disperso, es decir, cuando existen pocas conexiones en comparación con el total de nodos posibles.

La lista de adyacencia, en cambio, asocia a cada nodo una lista de sus vecinos directos junto con el peso de la conexión. Esta representación resulta más eficiente en memoria para grafos dispersos, como es el caso de la red de caminos de un campus universitario, motivo por el cual será la representación adoptada en este proyecto.

3.3 Algoritmos de Caminos Mínimos y el Algoritmo de Dijkstra

El problema de encontrar el camino más corto entre dos nodos de un grafo ponderado se conoce como el problema de caminos mínimos. Entre los algoritmos más utilizados para resolverlo se encuentran el algoritmo de Dijkstra, el algoritmo de Bellman-Ford y el algoritmo A*.

El algoritmo de Dijkstra, propuesto por Edsger W. Dijkstra en 1959, calcula la ruta más corta desde un nodo origen hacia todos los demás nodos de un grafo con pesos no negativos. Su funcionamiento se basa en mantener un conjunto de distancias provisionales desde el origen a cada nodo, seleccionando en cada iteración el nodo no visitado con la menor distancia acumulada y actualizando ("relajando") las distancias de sus vecinos si se encuentra un camino más corto a través de él. El proceso se repite hasta visitar todos los nodos alcanzables o hasta llegar al nodo destino.

Este algoritmo fue seleccionado para el proyecto porque garantiza encontrar la ruta óptima en grafos con pesos no negativos —condición que se cumple en este caso, ya que las distancias entre puntos del campus siempre son positivas— y porque, al implementarse con una cola de prioridad (min-heap), alcanza una complejidad de $O((V + E) \log V)$, adecuada para el tamaño de la red de caminos de un campus universitario.

3.4 Estructuras de Datos de Apoyo

Para la implementación eficiente del algoritmo de Dijkstra se emplea una cola de prioridad (priority queue), estructura que permite extraer en cada paso el nodo con menor distancia acumulada en tiempo logarítmico. Asimismo, se utilizan estructuras de mapeo (tablas hash) para asociar de forma eficiente los nombres de los lugares con los nodos del grafo y los edificios a los que pertenecen, lo cual soporta la funcionalidad de búsqueda directa por lugar.

3.5 Accesibilidad y Diseño Inclusivo

El diseño inclusivo busca que los productos y servicios puedan ser utilizados por la mayor cantidad de personas posible, incluyendo aquellas con movilidad reducida. En el contexto de este proyecto, este principio se traduce en la posibilidad de excluir del cálculo de rutas aquellos caminos que incluyan escaleras, priorizando alternativas con rampas o ascensores, de forma que el sistema sea utilizable por toda la comunidad universitaria.

3.6 Interacción Humano-Computador (IHC)

La interacción humano-computador estudia el diseño de interfaces que faciliten la comunicación entre el usuario y el sistema. Principios como la simplicidad, la retroalimentación visual inmediata y la consistencia son aplicados en el diseño de la interfaz gráfica del proyecto, con el fin de que la selección de origen, destino y la visualización de la ruta resulten claras e intuitivas para cualquier usuario.

4. Marco Tecnológico

A continuación se describen las tecnologías, herramientas y componentes que se emplearán para el desarrollo del sistema.

4.1 Lenguaje de Programación

El sistema se desarrollará en Java, un lenguaje orientado a objetos, multiplataforma y ampliamente utilizado en el ámbito académico y profesional. Java permite estructurar el sistema de forma modular (clases para el grafo, los nodos, las aristas, el algoritmo de Dijkstra y la interfaz gráfica), y su ejecución sobre la máquina virtual de Java (JVM) garantiza portabilidad entre distintos sistemas operativos.

4.2 Interfaz Gráfica: JavaFX

Para el desarrollo de la interfaz gráfica se utilizará JavaFX, framework moderno de Java para la construcción de aplicaciones de escritorio con interfaces enriquecidas. JavaFX permitirá implementar el mapa interactivo o la lista desplegable para la selección de origen y destino, así como la visualización gráfica de la ruta calculada sobre una representación del campus.

4.3 Estructuras de Datos y Algoritmos

- Grafo: implementado mediante una lista de adyacencia, utilizando estructuras del tipo Map<Nodo, List<Arista>> del paquete java.util.
- Cola de prioridad: la clase PriorityQueue de Java se utilizará para la implementación eficiente del algoritmo de Dijkstra.
- Colecciones auxiliares: HashMap y HashSet para el mapeo de lugares por edificio y el control de nodos visitados.

4.4 Persistencia de Datos

La información del grafo (edificios, lugares, caminos y sus pesos) se almacenará en archivos de datos (formato JSON o CSV), lo que permitirá cargar y actualizar la red del campus sin necesidad de modificar el código fuente. Esta decisión podrá evolucionar hacia el uso de una base de datos embebida (por ejemplo, SQLite) si el proyecto lo requiere en fases posteriores.

4.5 Herramientas de Desarrollo

- Entorno de desarrollo integrado (IDE): IntelliJ IDEA.
- Control de versiones: Git, con repositorio remoto alojado en GitHub.
- Herramientas de diagramación: draw.io / Lucidchart, para la elaboración de diagramas de clases, casos de uso y arquitectura en fases posteriores.
- Gestión del proyecto: metodología SCRUM, con tablero de tareas (por ejemplo, Trello o GitHub Projects) para el seguimiento de las iteraciones.
- Documentación: Microsoft Word y PDF para la entrega de los documentos técnicos de cada fase.

4.6 Justificación de la Selección Tecnológica

Java y JavaFX fueron seleccionados por tratarse de tecnologías robustas, orientadas a objetos y ampliamente documentadas, que facilitan tanto la implementación de estructuras de datos como grafos y colas de prioridad, como el desarrollo de interfaces gráficas de escritorio interactivas. Adicionalmente, su carácter multiplataforma garantiza que el sistema pueda ejecutarse en los distintos equipos utilizados por la comunidad universitaria, cumpliendo con el requerimiento de portabilidad definido para el proyecto.

